ColdTrace: Weekly Plan and Checklists

Nov 11, 2026 · @Muedi

The team has 17 days to turn a working detector into a deployed, secured, reviewable system. The final submission closes **Sat 28 Nov 2026, 11:59 PM Eastern, which is Sun 29 Nov 06:59 SAST**, so the plan finishes its own work by **Fri 27 Nov 18:00 SAST**.

How to read this plan

- Weeks run Monday to Sunday. Week 2 is already under way (today is Wed 11 Nov); the plan covers the rest of it, then Weeks 3 and 4.
- Assumption: the 10 Nov Week 2 deadline (backend model shared with the NVIDIA model) is treated as met, with one item still open: a clean rerun of the NIM comparison.
- Each person has one checklist per week. Tick items as they finish. An item is done only when its proof exists (a file, a passing test, a URL).
- Each week ends with a gate. If the gate fails, the next week starts by fixing it, not by adding scope.

Person	Lane	Owns
Muedi	AI research foundations and team facilitation	Detection model, baselines, evaluation, NIM incident notes, the Newly app build
Chantell	Data validation	Schema and quality checks, quarantine of bad events, BigQuery slice
Moorthy	Integration	GCP services, CI/CD, Cloud Run, Pub/Sub, Firestore, API endpoints
Peter	Cybersecurity	Secrets, access control, evidence integrity, vulnerability checks

End state on 27 Nov: a simulated reading from any of 3 locations flows through ingest, detection, an alert in Firestore, a human review in the app, and a closed case, with the evidence hash checked at every handoff.

Week 2 (remaining): Wed 11 to Sun 15 Nov

The goal is a backend everyone can run, in the team repo, with clean data checks and a security plan on paper. **Gate (Sat 14 Nov):** tests pass in CI, the API runs locally and `scripts/replay.py` produces alerts, and the GCP project exists.

Muedi

- [ ] Rerun `python scripts/compare_nim.py --per-class 30 --workers 1` and keep the table
- [ ] Choose the model for incident notes from the rerun and set it as `NIM_MODEL`; record why
- [ ] Write one page for the handoff: baselines (threshold, rules, random forest), event recall, time to detect, false alerts per location-day, and what the LLM may and may not be trusted for
- [ ] Push the backend (code, tests, README, no data file, no keys) to the team branch
- [ ] Ask Newly in Ask mode: which framework, whether it builds for web, whether GitHub sync works both ways; record the answers
- [ ] Request GitHub access for the team and make the first push

Chantell

- [ ] Run schema checks on `data/sample_14d.csv`, then the full file: value ranges, 30-minute cadence, no duplicate `(location_id, ts)`, no frozen runs
- [ ] Write a short data validation report: checks run, rows passed, rows failed, examples
- [ ] Confirm with the team that the three exception types (`TEMP_EXCURSION`, `COOLING_FAILURE`, `SENSOR_DROPOUT`) are the agreed set; the handbook does not name them
- [ ] Choose the telemetry slice for BigQuery (for example the 2026 test split, not all rows)
- [ ] Package the checks as a function the ingest endpoint can call

Moorthy

- [ ] Create the GCP project; enable Cloud Run, Pub/Sub, Firestore, Artifact Registry, Secret Manager, Logging and Monitoring
- [ ] Set up the repo structure and a GitHub Actions workflow that runs the tests on every push
- [ ] Run the API locally (`uvicorn coldtrace.api:app`) and replay 14 days against it
- [ ] Design the Firestore collections for alerts, reviews and audit events from `Alert.to_case_json()`
- [ ] List the API endpoints the app needs beyond the three that exist (see the Newly brief, section 6) and agree an order with Muedi

Peter

- [ ] Write a one-page threat model: assets (evidence, NIM key, ingest key), entry points, who can do what
- [ ] Plan Secret Manager: secret names, which service account reads each, no key in code or CI logs
- [ ] Review `coldtrace/api.py`: key comparison, per-alert access, rate limiting, payload size limits
- [ ] Add secret scanning (for example gitleaks) and a dependency scan to the CI workflow
- [ ] Confirm `.gitignore` covers `.nim_keys.json` and any `.env` file

All

- [ ] 15-minute check-in each day: done, next, blocked
- [ ] Review the gate on Sat 14 Nov and record what failed

Week 3: Mon 16 to Sun 22 Nov

The goal is a deployed, secured system with a working app on mock data. **Gate (Sat 21 Nov):** the Cloud Run URL answers `/healthz`; a simulated reading becomes an alert stored in Firestore; secrets come only from Secret Manager; the app runs phases 0 to 4 on the mock adapter.

Muedi

- [ ] Put the chosen NIM model behind `/alerts/{id}/explain` on Cloud Run, with the key read from Secret Manager
- [ ] Read 20 generated incident notes by hand: no invented facts, no invented batch IDs, `needs_human_review` set sensibly; record failures
- [ ] Re-score the detector on Chantell's validated data and note any change in recall or false alerts
- [ ] Build the Newly app phases 0 to 4 from the brief on the mock adapter, one phase per prompt, checking each phase's acceptance list
- [ ] Add the Detection Quality figures (with the synthetic-data notice) for the app

Chantell

- [ ] Add validation at ingest: out-of-range, duplicate and out-of-order readings go to a quarantine list, not to the detector

- [ ] Write test cases for bad data: duplicates, gaps longer than 30 minutes, frozen values, impossible temperatures
- [ ] Load the agreed slice into BigQuery and run three sample queries (alerts per day, readings per location, dropout hours)
- [ ] Report the quarantine counts for a full replay

Moorthy

- [ ] Deploy the API to Cloud Run through GitHub Actions and Artifact Registry; set max instances to 1 so each location's rolling buffer stays in one process
- [ ] Create the Pub/Sub topic and subscriber; add a Cloud Run Job or Scheduler that streams simulated readings for all 3 locations
- [ ] Persist alerts, reviews and audit events in Firestore, so a restart loses nothing
- [ ] Build the first endpoints: list locations, readings, alerts, one alert, acknowledge and close
- [ ] Turn on Cloud Logging and one alert policy (for example error rate)

Peter

- [ ] Wire Secret Manager for the NIM key and ingest key; grant each service account only the secrets it needs
- [ ] Add role checks to every endpoint and confirm each role against each endpoint in a test
- [ ] Add rate limiting and a request size limit
- [ ] Test evidence integrity: change one evidence row in Firestore and confirm the verify step reports a mismatch
- [ ] Run the vulnerability checklist: missing key, wrong key, oversized body, injection in text fields, replayed requests; log each result

All

- [ ] Daily 15-minute check-in
- [ ] Mid-week demo on Wed 18 Nov: one reading through to an alert on the deployed URL
- [ ] Gate review on Sat 21 Nov; anything failing becomes Monday's first task

Week 4: Mon 23 to Sat 28 Nov

The goal is a complete, tested loop and a clean submission. **Code freeze Thu 26 Nov 18:00 SAST. Everything done by Fri 27 Nov 18:00 SAST.** Saturday is buffer only, with no new work.

Muedi

- [ ] Finish Newly app phases 5 to 7 (Settings, Demo Simulator, hardening)
- [ ] With Moorthy, switch the app from the mock adapter to the live API through the Newly Backend proxy (phase 6)
- [ ] Run the full demo script: detection, human review, closed case, with a tampered-evidence example; time it
- [ ] Update the AI foundations page with final numbers and the limits (synthetic data, LLM not used for detection)
- [ ] Prepare the PR description and check the final README

Chantell

- [ ] Run the full validation suite against the deployed system and attach the results
- [ ] Replay a full week of simulated data end to end; confirm row counts match in ingest, Firestore and BigQuery
- [ ] Review the README data section: sources, checks, known gaps
- [ ] Final check that no real or personal data is in the repo

Moorthy

- [ ] Finish the remaining endpoints (batches at risk, detection metrics, audit, demo replay)
- [ ] Run a cold-start test: stop all instances, send a reading, confirm the alert still appears
- [ ] Write the deployment steps in the README so a stranger can reproduce the deploy
- [ ] Open the PR from the team branch to `main`; do not merge
- [ ] Confirm the CI run on the PR is green

Peter

- [ ] Re-run the Week 3 vulnerability checklist against the final deployment
- [ ] Scan the repo history for secrets and confirm none appear
- [ ] Review IAM: remove any role not in use
- [ ] Write a short security summary: what is protected, how, what is not covered
- [ ] Sign off in writing before the code freeze, or list what blocks sign-off

All

- [ ] Mon 23 Nov: wiring day, app on the live API
- [ ] Tue 24 Nov: end-to-end test run; log every defect

- [ ] Wed 25 Nov: fix defects; Peter's checks
- [ ] Thu 26 Nov: freeze at 18:00 SAST
- [ ] Fri 27 Nov: submission checklist below, finished by 18:00 SAST
- [ ] Sat 28 Nov: buffer only; submit well before 11:59 PM Eastern

Handoffs, risks and decisions

Every handoff has a proof the receiver can check, so a late or wrong handoff shows up the same day.

From	To	What	Proof it arrived
Muedi	Moorthy	Backend code, model file, README	Tests pass in CI; `replay.py` prints alerts
Chantell	Muedi, Moorthy	Validation function and report	Bad rows land in quarantine; counts match
Moorthy	Peter	Deployed service and IAM roles	`/healthz` answers; role tests pass
Peter	Muedi	Secret Manager wiring	Explain endpoint works with no key in code
Moorthy	Muedi	Live API endpoints	App runs against live adapter
Muedi	All	Newly app and demo script	Full loop runs once, start to finish

Risk	Effect	Response	Owner
Cloud Run restarts lose alert state	Alerts vanish mid-demo	Firestore persistence in Week 3; cold-start test in Week 4	Moorthy
More than one instance splits a location's buffer	Missed or false alerts	Max instances 1 for the MVP	Moorthy
Newly cannot build for web, or its GitHub sync fails	App does not match the plan	Ask Newly in Week 2; fall back to a small React page on Cloud Run or `newly pull`	Muedi
NIM rate limits or an outage	No incident notes	Detection never depends on NIM; show alert with no note and a clear message	Muedi

Risk	Effect	Response	Owner
LLM notes contain invented facts	Misleading review	Hand check 20 notes; keep the review label and human decision	Muedi
Synthetic data read as real accuracy	Overstated claims	State the limit in the README, app and write-up	Muedi
A key reaches the repo	Credential leak	Secret scan in CI; history scan in Week 4	Peter
Eastern deadline is overnight in SAST	Late submission	Internal freeze Fri 27 Nov 18:00 SAST	All

Decisions needed this week

- [ ] Is the dashboard a Newly mobile app, a React web page, or both? (Muedi, after asking Newly)
- [ ] Which NIM model writes incident notes? (Muedi, after the rerun)
- [ ] Which rows go to BigQuery? (Chantell and Moorthy)
- [ ] Who merges the PR after submission, and when? (Team lead)

Final submission checklist

Tick each item only when the proof exists. Target: all done by Fri 27 Nov 18:00 SAST.

- [ ] Repository: team branch holds everything; PR to `main` is open and not merged
- [ ] README: problem, architecture, how to run locally, how to deploy, environment variable names, data and validation, known limits
- [ ] CI: tests and secret scan are green on the PR
- [ ] Deployment: Cloud Run URL answers `/healthz`; Pub/Sub, Firestore and Secret Manager are in use
- [ ] Full loop: a simulated reading becomes an alert, is reviewed in the app, and the case is closed
- [ ] Evidence: each alert's hash is checked at every handoff; a tampered example is shown failing
- [ ] Data: validation report attached; bad rows are quarantined, not detected on
- [ ] AI: baselines and event metrics reported; NIM notes labelled "AI-generated, review before acting"; LLM not used for detection
- [ ] Honesty: synthetic-data limit stated in the README, the app and the write-up

- [ ] Security: threat model, vulnerability checklist results and Peter's written sign-off included
- [ ] Secrets: none in the repo, its history, CI logs or the app
- [ ] App: all ten screens open; roles enforced on the server; runs on the live adapter
- [ ] Demo: script and a timed run-through done by the team
- [ ] Submitted before Sat 28 Nov 11:59 PM Eastern (Sun 29 Nov 06:59 SAST), with the submission confirmation saved